嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑　楠

sn@uestc.edu.cn

2026年3月

第三部分

嵌入式操作系统基础

—— 嵌入式软件基础

核心内容

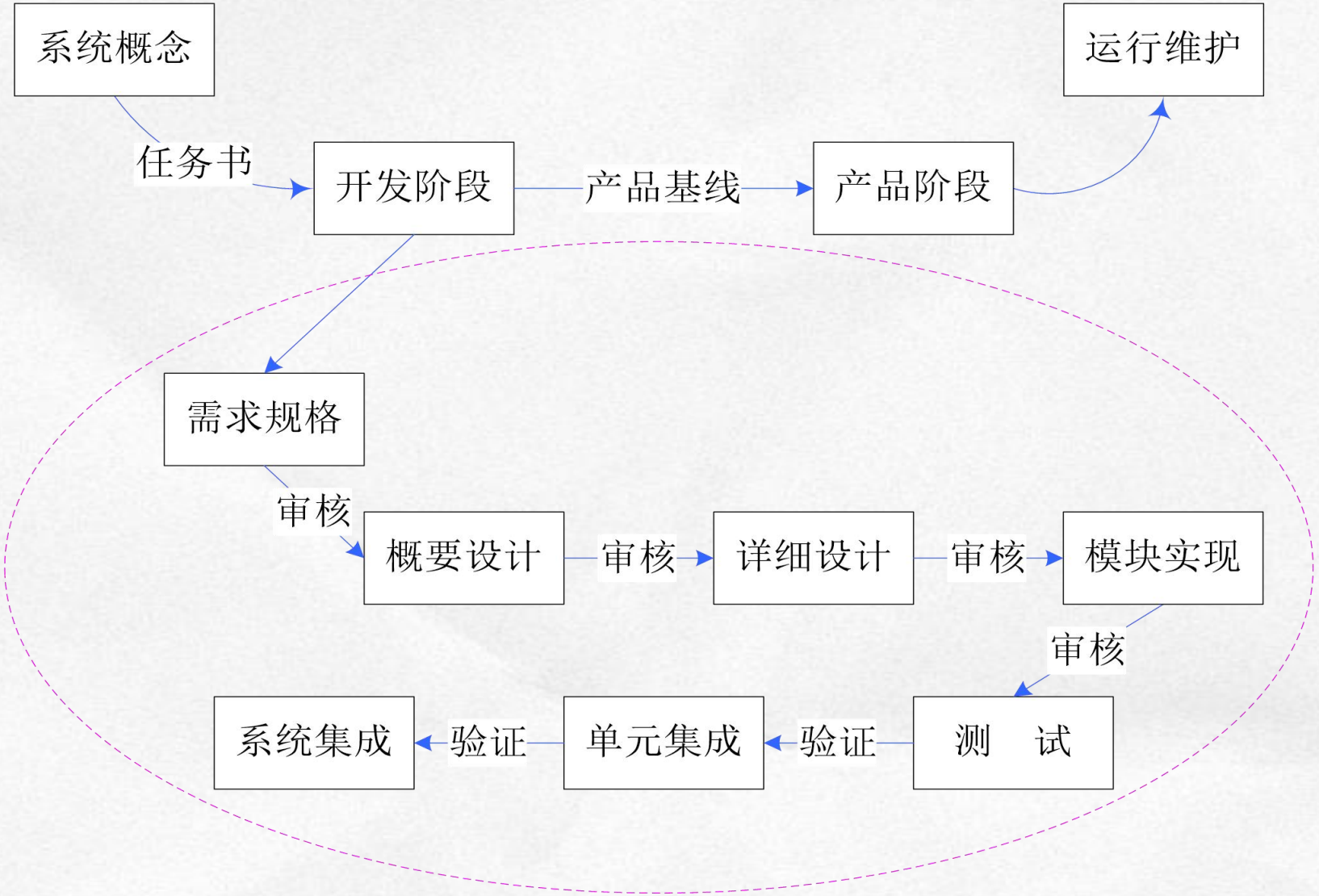
- 嵌入式软件系统及其开发
- 嵌入式软件的运行过程
- 系统引导
- 设备驱动
- 利用生成式AI编写驱动程序
- 嵌入式系统的快速启动

嵌入式软件系统基础

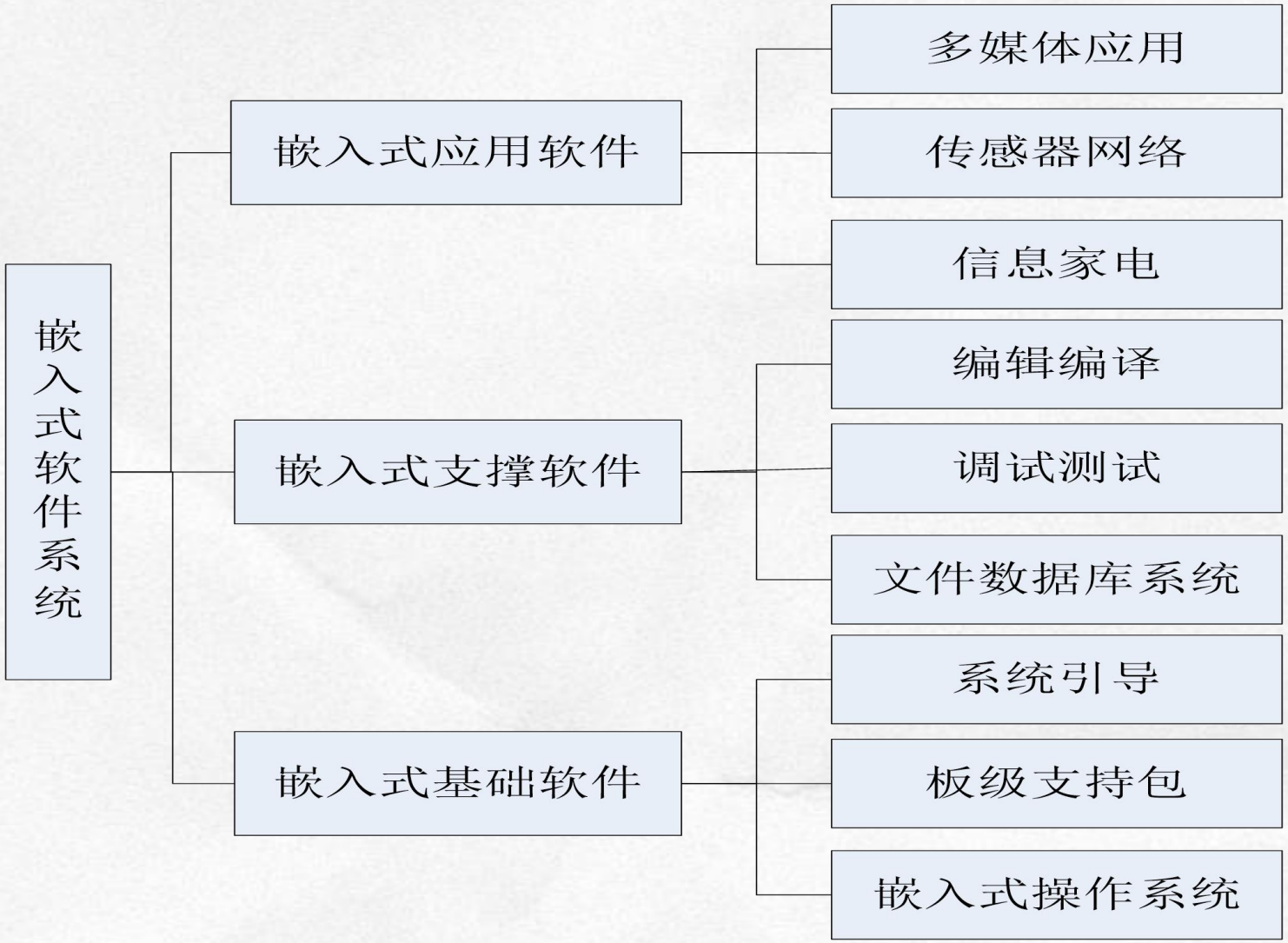
基本概念

- 软件　　算法＋数据结构＋文档；软件即服务
- 实时软件　　Reaitime Software，RTS
- 嵌入式软件　Embedded Software，ES
- 板级支持包　Board Support Package，BSP
- 启动加载　　Bootloader
- 驱动程序　　Driver

嵌入式软件的生命期


系统概念
任务书
开发阶段
产品基线
产品阶段
运行维护
需求规格
审核
概要设计
审核
详细设计
审核
模块实现
审核
测　试
验证
单元集成
验证
系统集成

嵌入式软件分类


嵌入式软件系统
嵌入式应用软件
多媒体应用
传感器网络
信息家电
嵌入式支撑软件
编辑编译
调试测试
文件数据库系统
嵌入式基础软件
系统引导
板级支持包
嵌入式操作系统

嵌入式软件的特点

- 实时性　　环境特定；快速启动
- 异步事件的并发处理　多任务；随机性
- 应用／操作系统一体化　VxWorks；Linux
- 应用固化　　不可修改性
- 实用性　面向行业；定向开发
- 适用性　可裁减，适应应用
- 鲁棒性　容错
- 够用即可　成本、资源
- 可信性　安全、防危、可靠等

嵌入式实时软件的关注点

- 任务的实时性
- 时间特性　　强调计算状态的时间边界和它们之间的关系
- 时间表达　　基于时间点、基于时间间歇（开始、终止）
- 时间约束　　在满足了所有启动条件后，对一个可执行对象开始执行和必须终止的约束：start、dealine

多任务软件系统的关注点

- 并发任务之间的协作关系
  - 互斥问题
  - 同步问题
- 信息隐藏
  - 任务之间、模块之间接口
  - 消除全局变量

理解

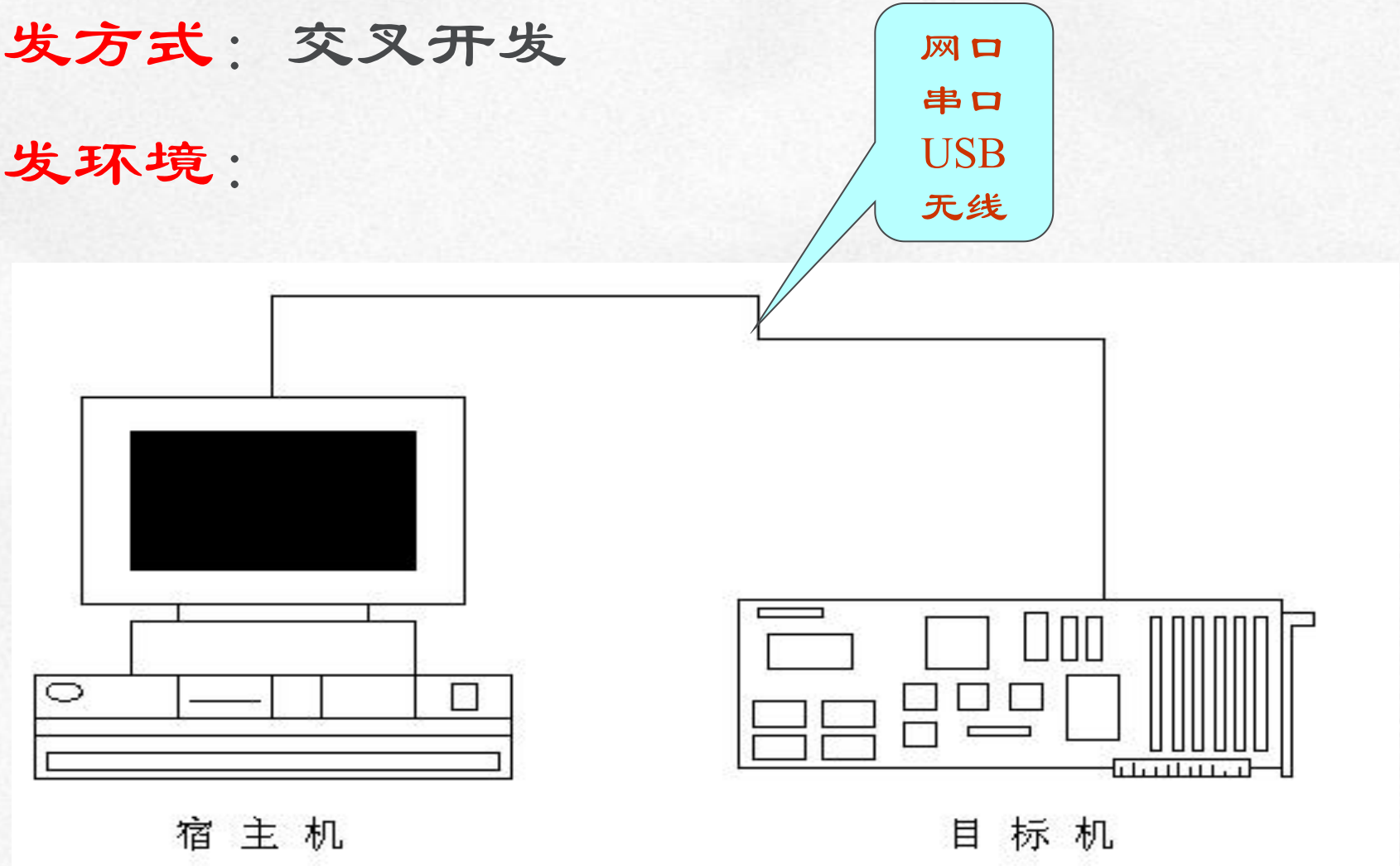
嵌入式系统开发

观点 —— 产品开发

- 内容：涉及H/W和S/W的复杂系统开发
- 特征：开发一个ES，相当于开发一台计算机
  - 需要开发硬件系统H/W
  - 对所开发硬件，需要开发/配置相应的驱动程序
  - 需要提供系统引导程序Bootload
  - 可能需要配置RTOS
  - 需要针对应用领域，采用专用的RTOS并进一步优化
  - 最后：开发特定领域的嵌入式应用软件

开发方式及开发环境

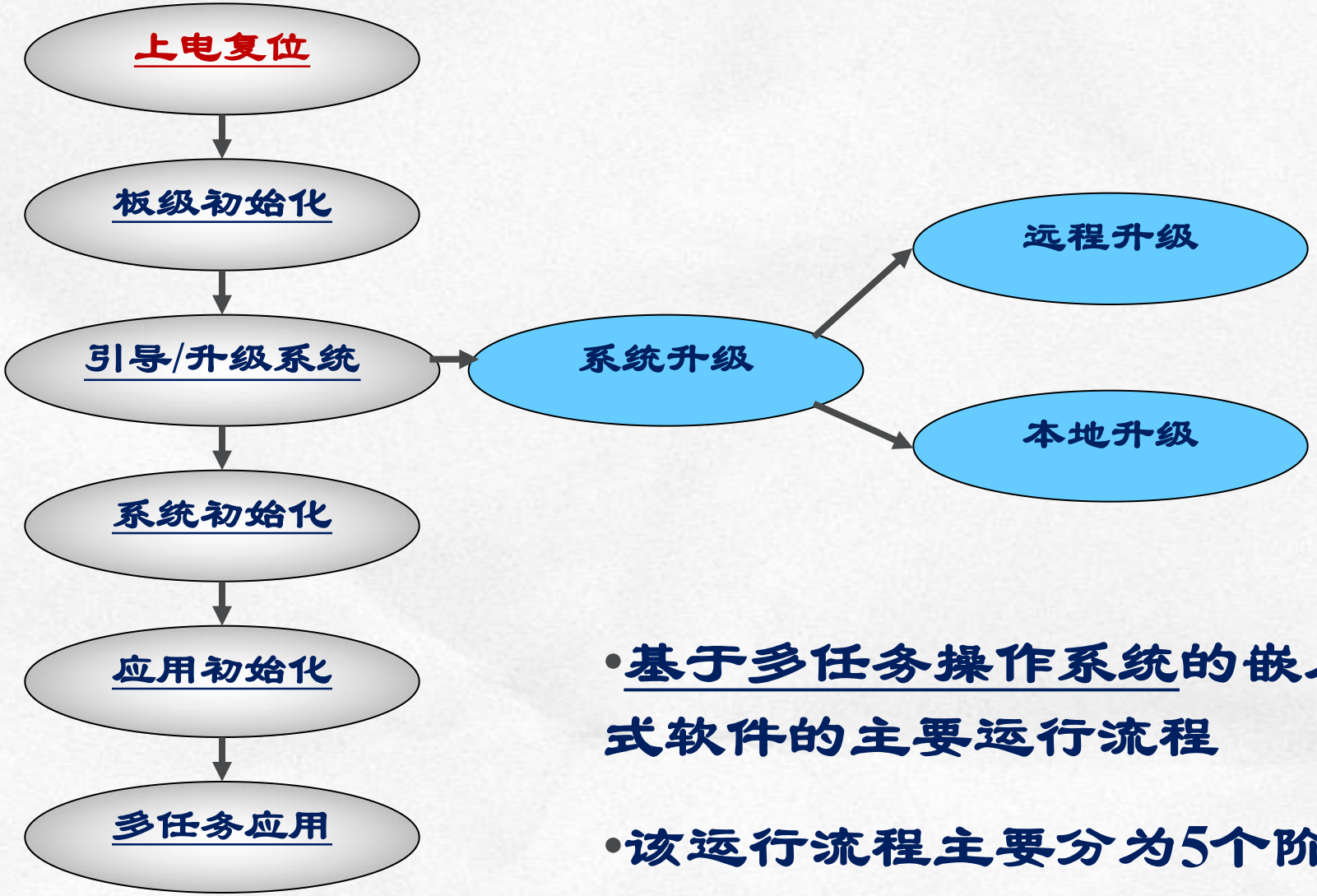
- 开发方式：交叉开发
- 开发环境：



嵌入式软件

运行过程

运行过程示意图



- 基于多任务操作系统的嵌入式软件的主要运行流程
- 该运行流程主要分为5个阶段

运行过程 —— 板级初始化

- **顺序**：上电复位后，开始板级初始化
- **特性**：完全的硬件相关，一般采用汇编语言实现
- **核心内容**：不同系统有所差异，但必须完成
  - CPU中堆栈指针寄存器的初始化。
  - BSS段（Block Storage Space）的初始化。
  - CPU芯片级的初始化：中断控制器、内存等的初始化。

运行过程 —— 系统引导

- **选择：** 根据需要分别进入系统软件引导阶段或系统升级阶段 —— 测试通信端口数据或判断特定开关
- **系统引导阶段：**
  - **RAM启动** 系统软件从NorFlash中读出并加载到RAM中运行：运行速度较快；软件可压缩存储在Flash中
  - **NorFlash启动** 系统软件直接在NorFlash上运行
  - **外存启动** 软件中读取出来加载到RAM中运行：成本低
- **系统升级阶段：**
  - **远程升级**：通过网络；支持TFTP、FTP、HTTP等方式
  - **本地升级**：使用超级终端或特定的升级软件进行。

运行过程 —— 系统初始化

- 操作系统等系统软件运行必需的初始化工作，如根据系统配置
  - 初始化数据空间
  - 初始化系统所需的接口和外设
  - ……
- 需要按特定顺序进行
  - 首先完成内核的初始化
  - 完成网络、文件系统等的初始化
  - 最后完成中间件等的初始化工作

运行过程 —— 应用初始化

- 应用软件运行必需的初始化工作，如
  - 应用任务的创建
  - 信号量的创建
  - 消息队列的创建
  - 应用相关的其它初始化
  - ……

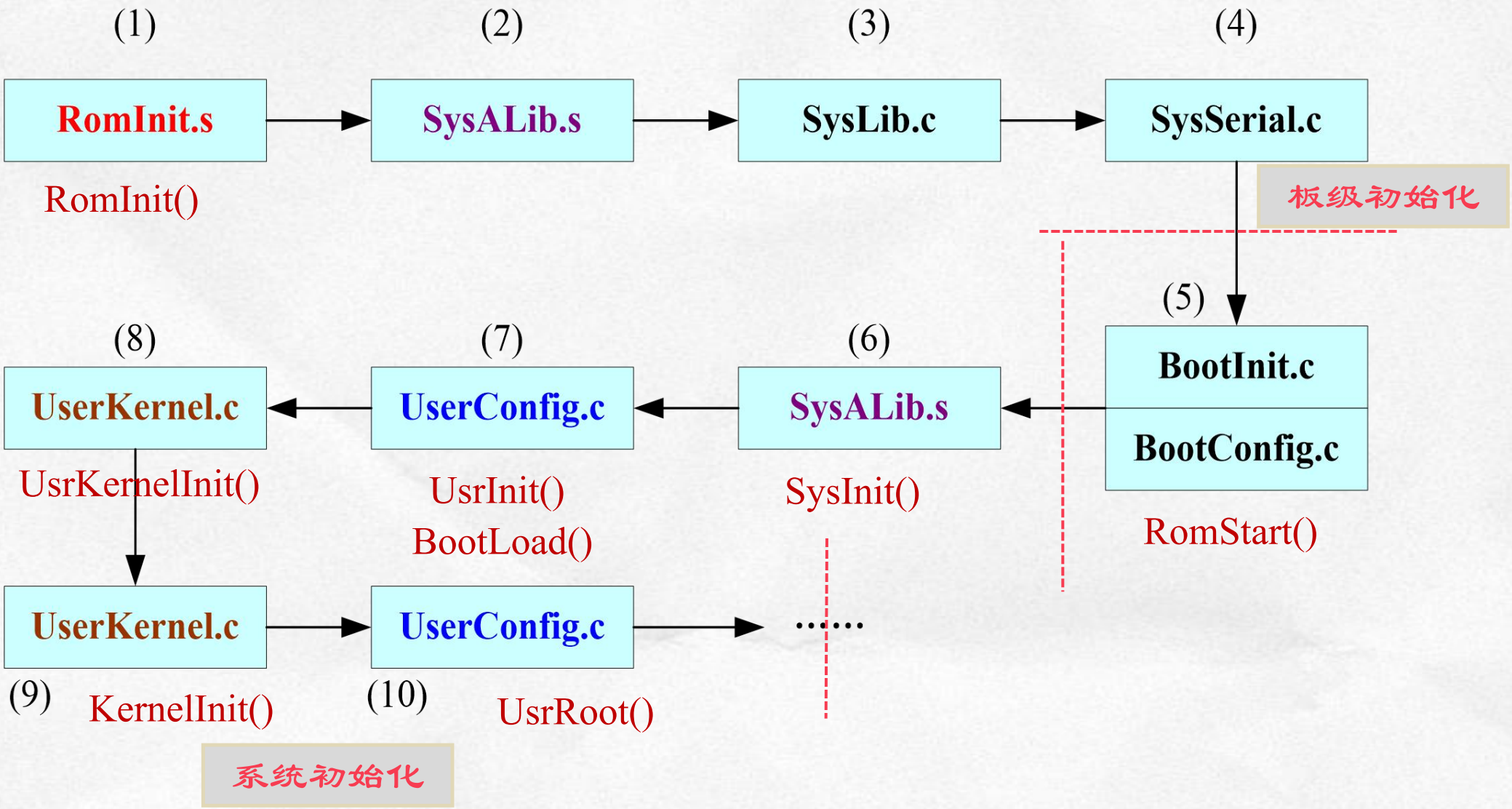
运行过程 —— 多任务系统运行

- 系统进入多任务状态
  - 操作系统按照已确定的算法进行任务的调度
  - 各应用任务分别完成特定的功能

运行过程 —— 函数

- **RomInit()**：romInit.s，上电初始化
- **RomStart()**：bootInit.c，程序代码调入RAM启动执行
- **SysInit()**：sysALib.s，类似romInit.s，但不再初始化SDRAM
- **UserInit()**：bootConfig.c，启动操作系统内核必须的初始化，C代码
- **BootLoad()**：bootConfig.c，加载操作系统映像（vxWorks）
- **UsrKernelInit()**：usrKernel.c，初始化内核可选组件
- **KernelInit()**：usrKernel.c，初始化多任务环境，启动内核
- **usrRoot()**：usrConfig.c，产生usrRoot根任务，初始化I/O系统、驱动器、设备等

运行过程 —— 程序代码示意


(1)
(2)
(3)
(4)
RomInit.s
SysALib.s
SysLib.c
SysSerial.c
RomInit()
板级初始化
(5)
(8)
(7)
(6)
BootInit.c
UserKernel.c
UserConfig.c
SysALib.s
BootConfig.c
UsrKernelInit()
UsrInit()
SysInit()
BootLoad()
RomStart()
UserKernel.c
UserConfig.c
……
(9)
KernelInit()
(10)
UsrRoot()
系统初始化

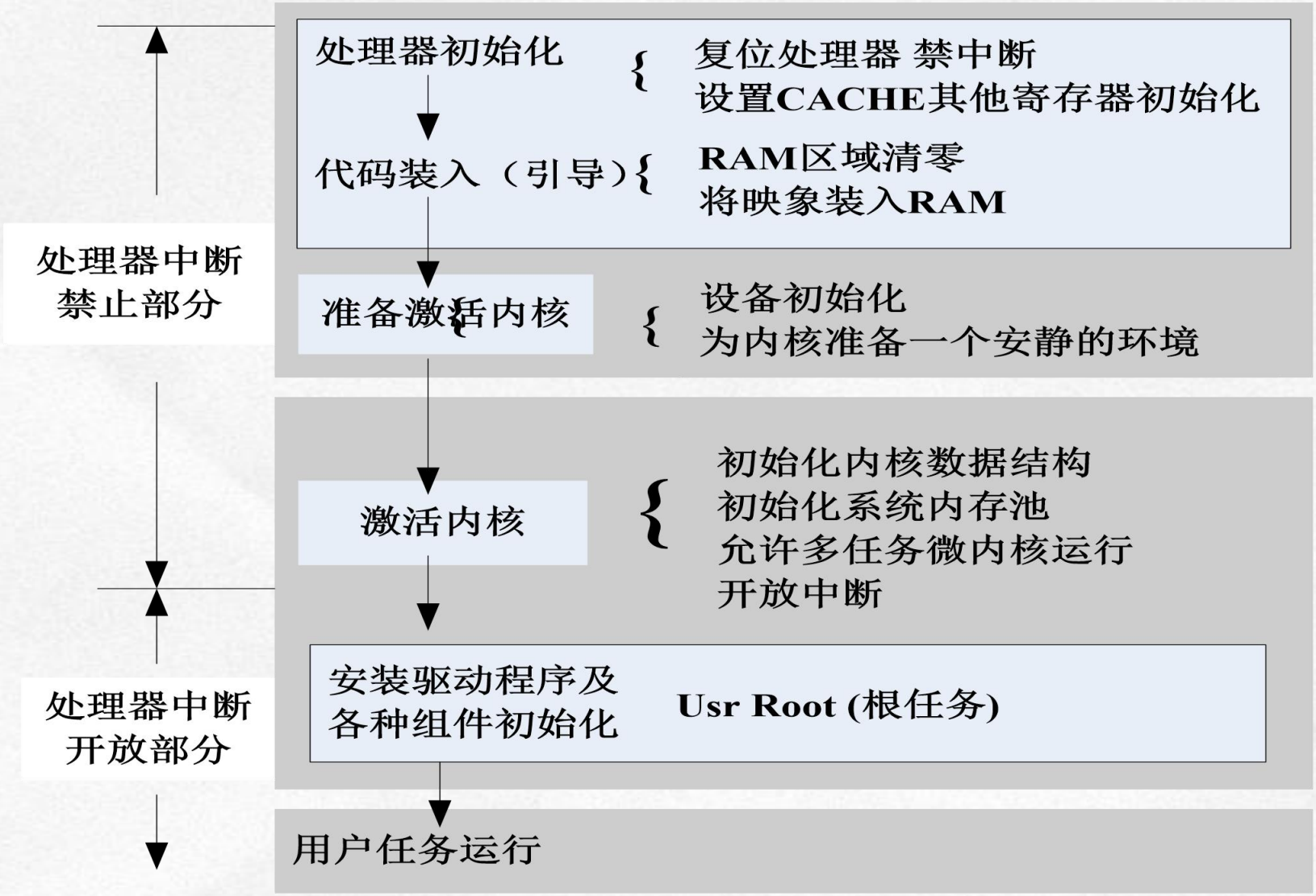
RTOS的引导模式

- 需要Boot Loader的引导模式　　采用高性能RAM的系统；受RAM空间的限制，运行时载入
  - 让PC跳转到刚拷贝到内存的内核的首条指令
- 不需要Boot Loader的引导模式　　对需要快速启动的系统；时间效率要求较高，空间限制较少。
- - 方法：直接在NorFlash或ROM等非易失性主存介质中运行，启动速度快

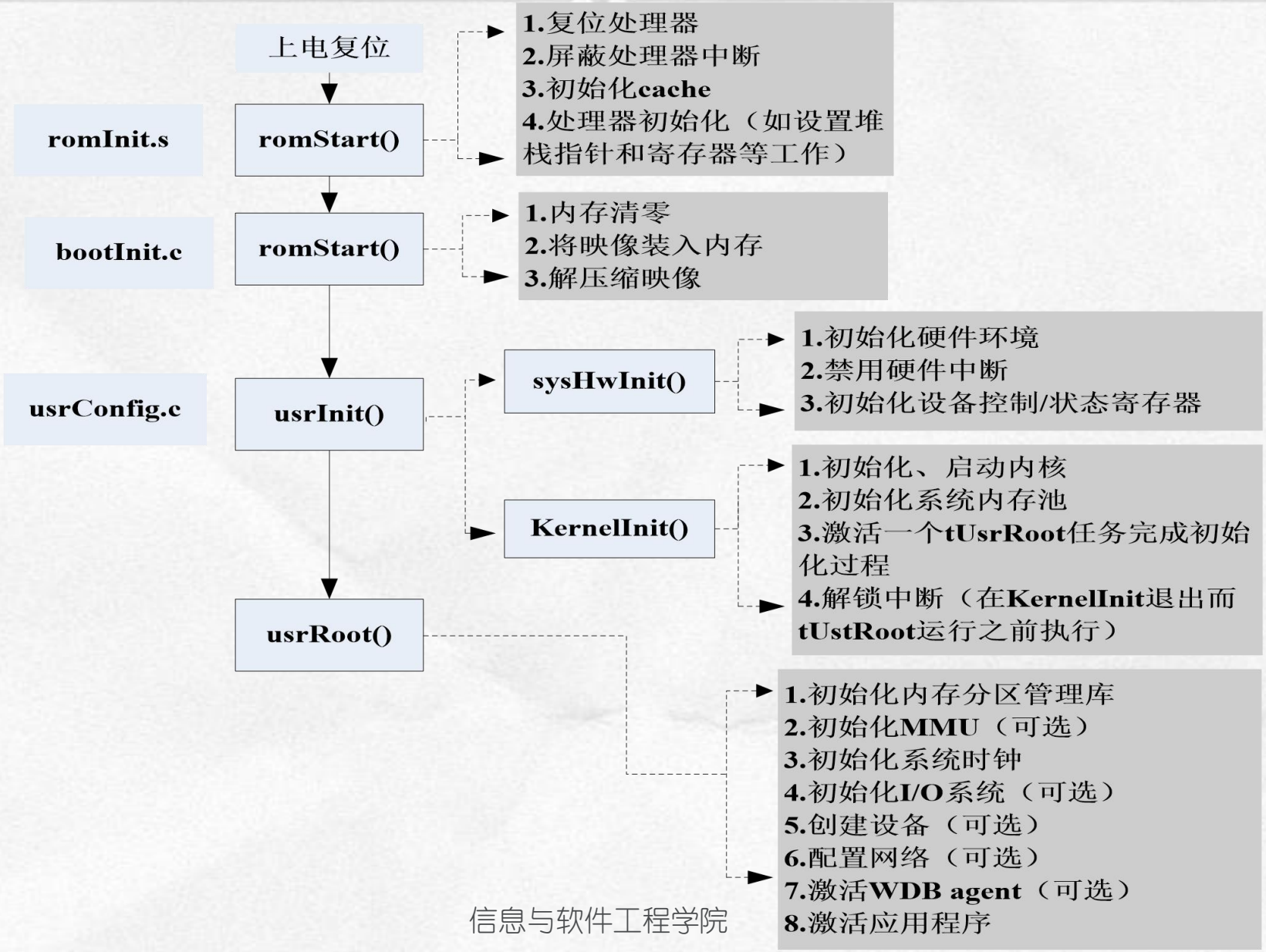
VxWorks

启动过程

基本启动过程


处理器初始化
复位处理器 禁中断
设置CACHE其他寄存器初始化
代码装入（引导）
RAM区域清零
将映象装入RAM
处理器中断
禁止部分
准备激活内核
设备初始化
为内核准备一个安静的环境
激活内核
初始化内核数据结构
初始化系统内存池
允许多任务微内核运行
开放中断
处理器中断
开放部分
安装驱动程序及
各种组件初始化
Usr Root (根任务)
用户任务运行

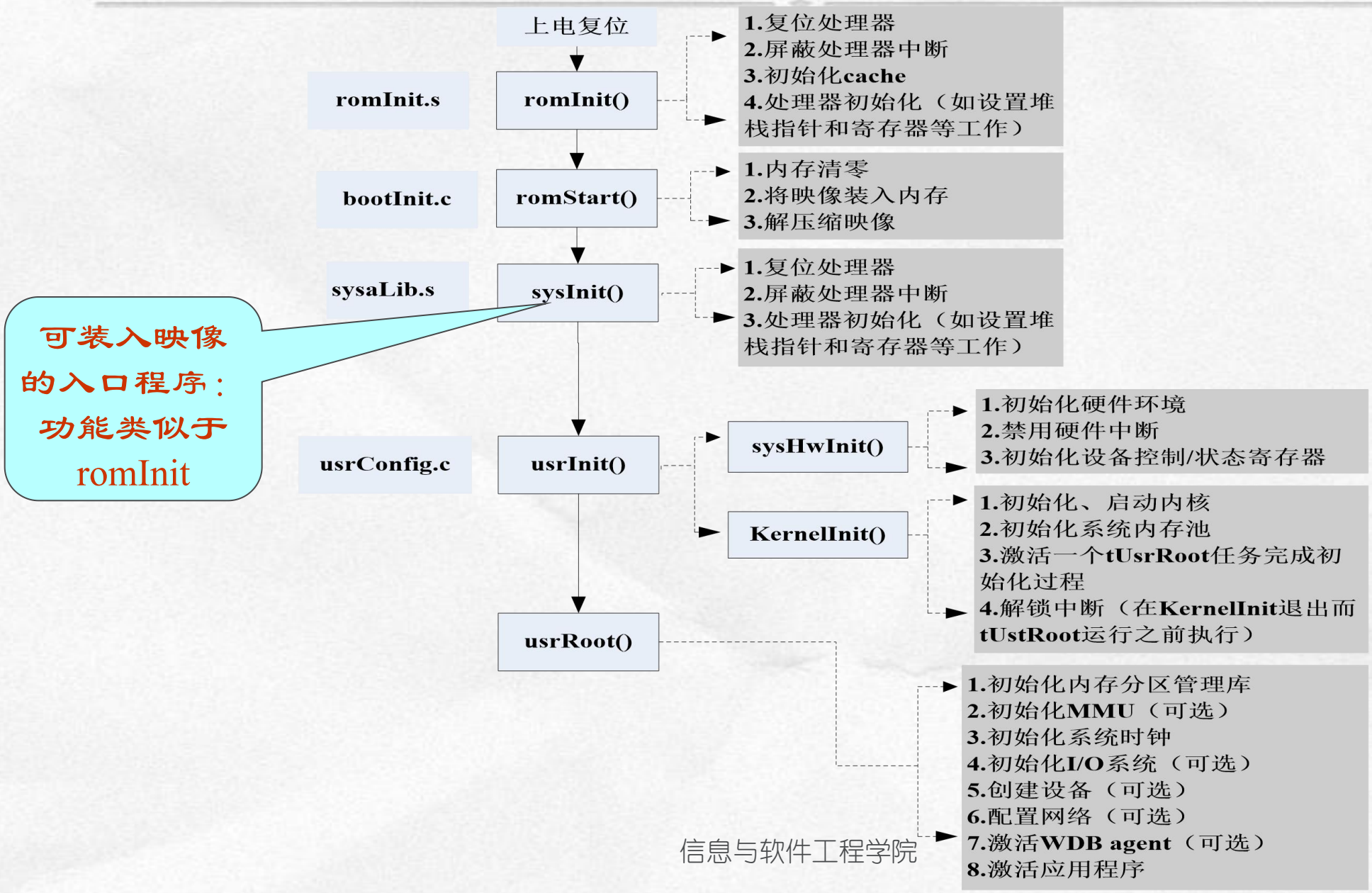
示例：ROM中引导VxWorks



示例：ROM中引导VxWorks（2）

- **存储运行模式**　　写入ROM中的VxWorks是非压缩的，不需要解压，系统直接跳到ROM的首地址，运行VxWorks
- **原因**　　VxWorks在ROM中运行主要是为了节省RAM空间，使应用程序有更大的空间运行
- **优缺点**　　ROM中运行的VxWorks优点是启动速度快，缺点是运行速度慢

示例：RAM中引导VxWorks



示例：RAM中引导VxWorks（2）

- **存储运行模式**　　写入ROM或Flash中的BootROM Image或VxWorks Image是压缩的，需要先拷贝、解压所有的text和data到RAM的RAM_LOW_ADRS中，然后运行VxWorks
- **优缺点**　　RAM中运行的VxWorks优点是运行速度快，但启动速度偏慢

aCoral

启动过程

启动程序入口点

```
ENTRY(__ENTRY)          // GNU程序规则
MEMORY
{
    RAM (wx) : org =0x30000000, len = 64M    // __ENTRY的起始地址
}
SECTIONS
{
        .text :
        {
                text_start = .;
                * (.text)                // 起始段：aCoral中 start.s编译后生成的目标文件
                * (.init.text)
                * (.rodata*)
        }>RAM
        …………………………………………………………
}
```

启动程序start.s

- 见Word文档

  教材7.3

板级初始化

板级初始化（Boot）的主要内容

- 处理函数：romInit()，上电后执行的第一个函数
- 处理过程：以vxWorks为例
  - 关中断
  - 设置哑堆栈（dummy stack，也称虚拟堆栈）
  - 保存启动类型（冷启动和热启动）到堆栈上
  - 清Cache
  - 初始化CPU内的寄存器
  - 硬件初始化，主要是初始化SDRAM接口
  - 调用romStart()

romInit()示例

```
        ……                                  // 定义等
        cli                                 /* 关中断 */
        movl    SP_ARG1(%esp),%ebx          /* esp+SP_ARG1的值给ebx：开始类型 */
        jmp     warm                        // 段内相对跳转到warm

        /* 切换到保护模式 */
        mov     %cr0,%eax                   /* CR0中含有系统控制标志 */
        .byte   0x66                        /* next inst has 32bit operand */
        or      $0x00000001,%eax            /* set the PE（保护使能） bit */
        mov     %eax,%cr0                   /* move EAX to CR0 */
        jmp     romInit1                    /* 跳转到romInit1 */
        ……
        /* 初始化 DS, ES, FS, GS, SS段寄存器 */
romInit1:
        mov     $0x0010,%eax                /* set data segment 0x10 is 3rd one */
        mov     %ax,%ds                     /* set DS */
        mov     %ax,%es                     /* set ES */
        mov     %ax,%fs                     /* set FS */
        mov     %ax,%gs                     /* set GS */
        mov     %ax,%ss                     /* set SS */
        .byte   0x66                        /* next inst has 32bit operand */
        mov     $ ROM_STACK,%esp            /* set lower mem stack pointer */
        ……
```

关中断： cli

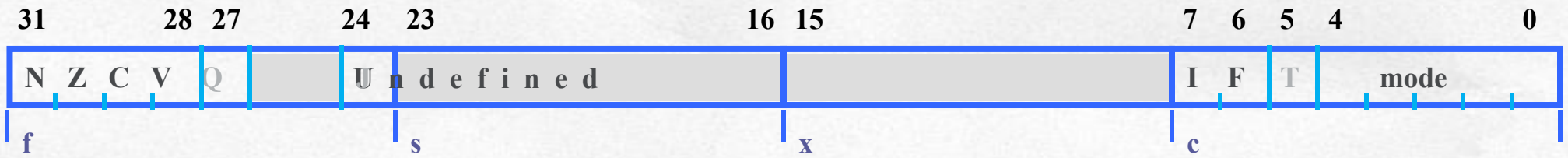
0x06000000

```
__ENTRY:
ResetHandler:
        MRS     r0, CPSR
        ORR     r0, r0, #(PSR_I_BIT|PSR_F_BIT)
        MSR     CPSR_c, r0
```

0x80　　0x40

@ Disable IRQ & FIQ

I、F位置1



系统引导

系统引导（Load）的主要内容

- 处理函数：romStart()，执行的第一个C程序
- 处理过程：
  - 完成必要的硬件初始化：sysHwInit()
  - 把ROM代码和数据段拷贝并重定位到RAM
    - 代码段未驻留在ROM中，拷贝代码段和数据段
    - 代码段是驻留在ROM中的，只拷贝数据段
  - 未用的内存(RAM)清零
  - 需要时，要进行解压缩
  - 跳转到sysInit()

这部分可以在以后处理：usrInit

romstart()示例

```
#ifdef  ROM_RESIDENT
        void romStart   ( FAST int startType      /* start type */   )
        {
                /* relocate the data segment into RAM */
                copyLongs ((UINT *)ROM_DATA_ADRS,
                        (UINT *)UNCACHED(RAM_DATA_ADRS),
                        ((UINT)end - (UINT)RAM_DATA_ADRS) / sizeof (long));

                /* If cold booting, clear memory to avoid parity errors */
                #ifdef ROMSTART_BOOT_CLEAR
                        if (startType & BOOT_CLEAR)
                                bootClear();
                #endif

                /* jump to the entry */
                sysInit (startType);
        }
#endif  /* ROM_RESIDENT */
```

这里可直接调用usrInit

sysHwInit()

- **功能**：硬件初始化
- **处理过程**：
  - 调用sysMemMapDetect()检测存储映射配置，相关全局变量赋值
  - 记录物理存储区的长度，计算sysPhysMemDesc的入口数
  - 调用pciInit()初始化PCI接口
  - 配置CPLD内的寄存器
  - 初始化EPIC、DMA等的相关寄存器的基本地址
  - 调用sysEpicInit()完成EPIC(段控制器)初始化
  - 调用sysSerialHwInit()初始化串口
  - 调用sysNetHwInit()初始化网络接口

串口初始化

```
LDR     r1, =UART0_BASE
MOV     r0, #0
STR     r0, [r1,#UART_PL011_CR]        @ Disable UART
MOV     r0, #0x27
STR     r0, [r1,#UART_PL011_IBRD]      @ Set speed to 38400bps
MOV     r0, #4
STR     r0, [r1,#UART_PL011_FBRD]
MOV     r0, #0x70
STR     r0, [r1,#UART_PL011_LCRH]      @ 8 bits, 1 stop bit, no parity, fifo enabled
LDR     r0, =0x301
STR     r0, [r1,#UART_PL011_CR]        @ Enable UART
```

系统初始化

系统初始化的主要内容

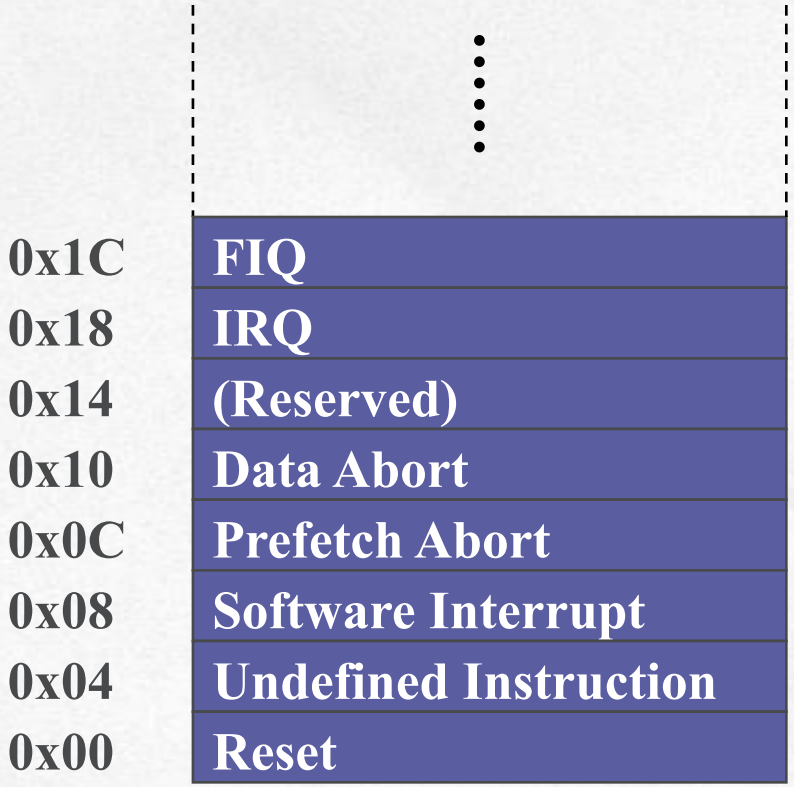
- **处理函数**：sysInit()，依赖特定系统的汇编代码模块，类似romInit()（不处理RAM）
- **处理过程**：
  - 关中断
  - 无效Cache
  - 初始化处理器的寄存器，包括C堆栈指针
  - 清除所有悬置的中断
  - 跳转到usrInit()

用户初始化

用户初始化的主要内容

- 处理函数：usrInit()，VxWorks下的第一个C程序
- 处理过程：
  - 无效Cache
  - BSS段清0
  - 保存启动类型
  - 调用函数intVecBaseSet()设置中断向量表基本地址
  - 调用sysHwInit()初始化硬件到静止状态：设备复位并禁止其产生中断
  - 调用usrKernelInit()，选择适当的初始化函数完成初始化
  - 调用kernelInit()，初始化多任务环境

中断向量表初始设置：VecBaseSet()




```
HAL_VECTR_START:
        LDR     pc, VECTOR_TABLE+0      @ Reset
        LDR     pc, VECTOR_TABLE+4      @ Undefined
        LDR     pc, VECTOR_TABLE+8      @ SWI
        LDR     pc, VECTOR_TABLE+0xc    @ Prefetch Abort
        LDR     pc, VECTOR_TABLE+0x10   @ Data Abort
        LDR     pc, VECTOR_TABLE+0x14   @ RESERVED
        LDR     pc, VECTOR_TABLE+0x18   @ IRQ
        LDR     pc, VECTOR_TABLE+0x1c   @ FIQ

VECTOR_TABLE:
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long EXP_HANDLER
            .long HAL_INTR_ENTRY
            .long EXP_HANDLER
HAL_VECTR_END:
```

usrKernelInit()

- **功能**：初始化内核可选组件
- **处理过程**：依次调用，处理过程可定制
  - classLibInit()
  - taskLibInit()
  - taskHookInit()
  - semBLibInit()
  - semMLibInit()
  - semCLibInit()
  - semOLibInit()
  - wdLibInit()
  - msgQLibInit()
  - qInit()
  - workQInit()

KernelInit()

- **功能**：初始化多任务环境，启动内核
- **处理过程**：
  - 激活intLockLevelSet() —— 设置分级中断管理
  - 使时间片（round-robin）方式无效 —— 针对实时
  - 从内存池顶部创建中断堆栈、根堆栈、以及TCB
  - 调用
    - taskInit()
    - taskActivate()
    - ……
  - 调用usrRoot()
  - 中断usrInit()的运行
  - 开中断

本部分涉及EOS的初始化，将在后续章节中细化

系统启动

系统启动的主要内容

- **处理函数**：usrRoot()，初始化I/O系统、驱动器、设备，启动系统
- **处理过程**：
  - 内存清零：memInit (pMemPoolStart, memPoolSize)；
  - 初始化MMU单元：usrMmuInit ();
  - 时钟设置：sysClkConnect()，sysClkRateSet()，sysClkEnable ()；
  - I/O初始化：iosInit () ，限制了最多同时打开设备数；
  - 字符设备设置：ttyDrv()，缺省为对控制台的调用；
  - 初始化结构、信号、消息等：excInit()，logInit()，sigInit()
  - 初始化管道等：pipeDrv()，stdioInit()，mathSoftInit()或mathHardInit()；
  - 初始化USB、SCSI、IDE、SATA、网络等设备接口；
  - 配置并初始化目标代理机：wdbConfig()
  - **启动内核**

板级支持包

BSP

BSP的概念

- 嵌入式系统的基础部分，实现系统可移植性的关键
- 负责
  - 上电时的硬件初始化、启动RTOS或应用程序的支持模块、提供底层硬件驱动
  - 提供上层软件提供访问底层硬件的手段

图6.2 嵌入式系统结构中的BSP

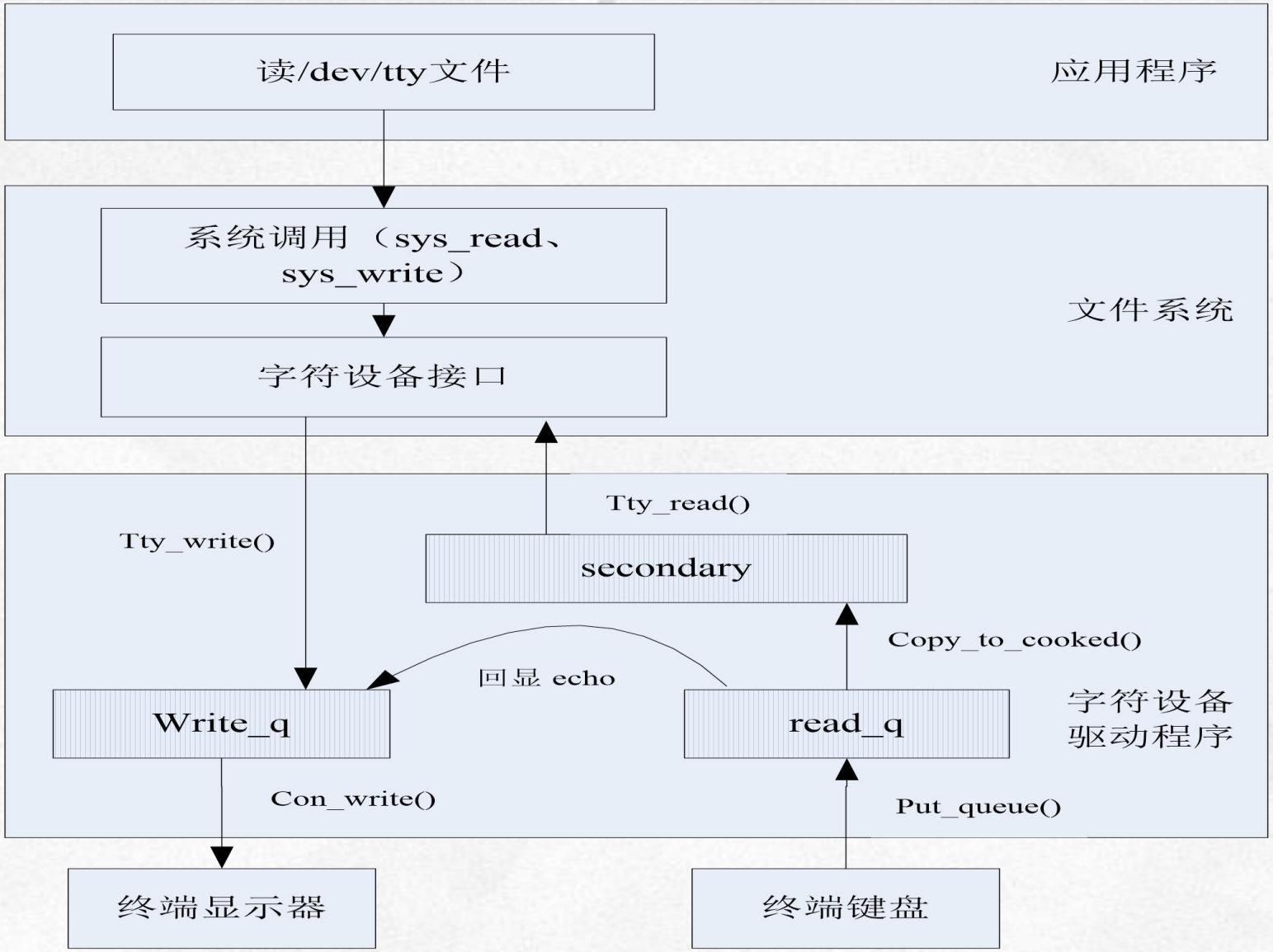
驱动程序

- **概念**　即设备驱动程序（Device Driver），一种可以使计算机和设备通信的特殊程序，相当于硬件的接口。
- **作用**　上层操作只有通过这个接口，才能控制硬件设备——“硬件和系统之间的桥梁”。
- **基本功能**
  - 对设备初始化和释放。
  - 对设备进行管理。
  - 读取应用程序传送给设备文件的数据，并回送应用程序请求的数据。
  - 检测和处理设备出现的错误

驱动程序的开发准备

- **了解** 硬件工作原理，包括处理器架构及工作原理、外设控制器的原理及接口（datasheet）。
- **对裸机** 驱动程序就是一套与硬件交互的函数库。
- **基于OS** **还必须**熟悉该OS对相关硬件的操作原理
- **驱动类别**
  - **字符设备** 以字节为单位顺序处理数据，不支持随机读写。如打印机、串口、显示器等
  - **块设备** I/O时数据处理以块为单位，采用缓冲，支持随机读写。如硬盘、光驱、U盘等。
  - **网络设备** 面向网络协议层，设备文件标识唯一，采用一套与数据包传输相关的函数。

示例：字符设备驱动程序运行过程



示例：VC++的串口通信

- **VC的通信控制函数**：
  - SetCommPort：指定使用的串口，如COM1、COM2、...
  - GetCommPort：获取当前使用的串口
  - SetSettings：设置串口的参数，如9600，N，8，1
  - GetSettings：取得串口参数
  - SetPortOpen：打开或关闭串口
  - SetInPutLen：一次读取输入缓冲区的字符数。0是全部字符。
  - GetInBufferCount：输入缓冲区中接受到的字符数
  - GetInPut：读取输入缓冲区
  - GetOutBufferCount：输出缓冲区中待发送的字符数
  - SetOutPut：写入输出缓冲区
  - ……

示例：VC++的串口通信（2）

- **利用控制函数的通信**：
  - 初始化：BOOL CMyDiaLog::OnIni
  - 定时串口读写：void CMyDiaLog::

```
BOOL CMyDiaLog::OnInitDialog()
{
CDialog::OnInitDialog();
m_Com.SetCommPort(1);
if (!m_Com.GetPortOpen()) {
m_Com.SetSettings("57600,N,8,1");
m_Com.SetPortOpen(true);
m_Com.SetInBufferCount(0);
SetTimer(1,10,NULL);
InBuffer.bstrVal=new unsigned short[MESSAGELENGTH];
OutBuffer.bstrVal=new unsigned short[MESSAGELENGTH];
OutBuffer.vt=VT_BSTR;
}
return true;
}
```

```
void CMyDiaLog::OnTimer(UINT nIDEvent)
{
    if (m_Com.GetInBufferCount()>=MESSAGELENGTH)
     {
            InBuffer=m_Com.GetInput();

            // handle the InBuffer.
            .....
            // Fill the OutBuffer.
            .....
            m_Com.SetOutput(OutBuffer);
     }
    CDialog::OnTimer(nIDEvent);
}
```

示例：VC++的串口通信（3）

- **直接用VC++访问串口**：使用的函数
  - CreateFile：用指定的方式打开指定的串口。
  - ReadFile：读取串口数据
  - WriteFile：向串口写数据
  - CloseHandle：关闭串口
- **串口通信的驱动函数**
  - 打开串口：BOOL CSerial::Open(int nPort, int nBaud)
  - 缓冲中字符数：int CSerial::InBufferCount(void)
  - 读数据：DWORD CSerial::ReadData(void *buffer, DWORD dwBytesRead)
  - 写数据：DWORD CSerial::SendData(const char *buffer, DWORD dwBytesWritten)

示例：VC++的串口通信（4）

- **写数据**

```
DWORD CSerial::SendData( const char *buffer, DWORD dwBytesWritten)
{
        if( !m_bOpened || m_hComDev == NULL ) return( 0 );
        BOOL bWriteStat;
        bWriteStat = WriteFile( m_hComDev, buffer, dwBytesWritten, &dwBytesWritten,
                                &m_OverlappedWrite );
        if( !bWriteStat){
                if ( GetLastError() == ERROR_IO_PENDING ) {
                        WaitForSingleObject( m_OverlappedWrite.hEvent, 1000 );
                        return dwBytesWritten;
                }
                return 0;
        }
        return dwBytesWritten;
}
```

利用生成式AI
编写驱动程序

基本思路

- **核心原则**：规范化
  - AI生成的代码质量高度依赖于输入指令的清晰度
  - 模糊的提示（如编写串口驱动程序）会导致不可靠的代码
- **明确硬件与环境**：AI不能替代对硬件的深入理解
  - 芯片架构：如Hi3559ARFCV100、RV64G（RISC-V 64位）
  - 硬件模块：指定的硬件，如TCU（张量计算单元）
  - 内存映射：关键寄存器的基地址和偏移量
  - 软件环境：指定软件环境，如HarmonyOS的驱动程序
- **定义功能与非功能需求**：参考软件工程需求分析
  - 功能需求：如初始化TCU模块，配置其支持INT8精度计算
  - 非功能需求：如初始化延迟必须小于10微秒
- **提供上下文与约束**：帮助AI理解背景，如类似硬件的现有代码片段

示例：RISC-V AI芯片TCU初始化

- **编写规范**：
  - **目标**：在RISC-V平台上，为张量计算单元（TCU）编写初始化驱动代码
  - **上下文**：TCU基地址为0x4000_0000，控制寄存器偏移为0x0，状态寄存器偏移为0x4。系统使用riscv64-unknown-elf-gcc编译器。目标为裸机程序
  - **功能需求**：设置串口的参数，如9600，N，8，1
    - ✓ 复位TCU：向控制寄存器写入0x0
    - ✓ 等待硬件就绪：轮询状态寄存器的BUSY_BIT（第0位），直到其为0
    - ✓ 启用TCU：向控制寄存器写入0x1
    - ✓ 开启中断：向控制寄存器写入0x2（假设第1位为中断使能位）
  - **约束**：必须使用volatile uint32_t*指针访问寄存器；代码需为C语言；不使用任何外部库

示例：RISC-V AI芯片TCU初始化（续一）

- **引导AI生成**：
  - 将上述规范输入给AI代码生成工具（如Cursor、GitHub Copilot等）
  - **进一步要求**："请生成符合上述规范的C语言代码，并添加必要的注释说明每一步的作用"
- **验证与迭代**：
  - **代码审查**：**人工/编译审查**AI生成的代码，检查寄存器地址、位操作、volatile关键字使用、时序逻辑等是否正确
  - **仿真与调试**：使用QEMU等RISC-V模拟器加载生成的代码，通过GDB调试，观察是否符合预期
  - **迭代优化**：若有误，**将错误信息和修正要求反馈给AI**，例如："在轮询状态寄存器时死循环，因为BUSY_BIT的定义错误，正确值应为0x1，请修正"

嵌入式系统

快速启动

现状与问题

- **现状：** 嵌入式系统功能越来越多 → **系统复杂、膨胀**
- **现状：** EOS不断升级，外设越来越多 → **启动变慢**
- 问题： **用户体验变差**
- **现状：** 关键应用对实时性要求很高
- 问题： **系统进入状态变慢**
- 如何解决？→ **快速启动**

影响启动速度的因素

- 处理器速度
- 存储设备　ROM、RAM、Flash、… → 速度、大小
- 硬件初始化　设备数量，设备初始化的效率
- 系统软件加载　内核、中间件、压缩、… → 启动项数、规模
- 应用软件加载　针对直接到应用主界面；~~服务数量~~

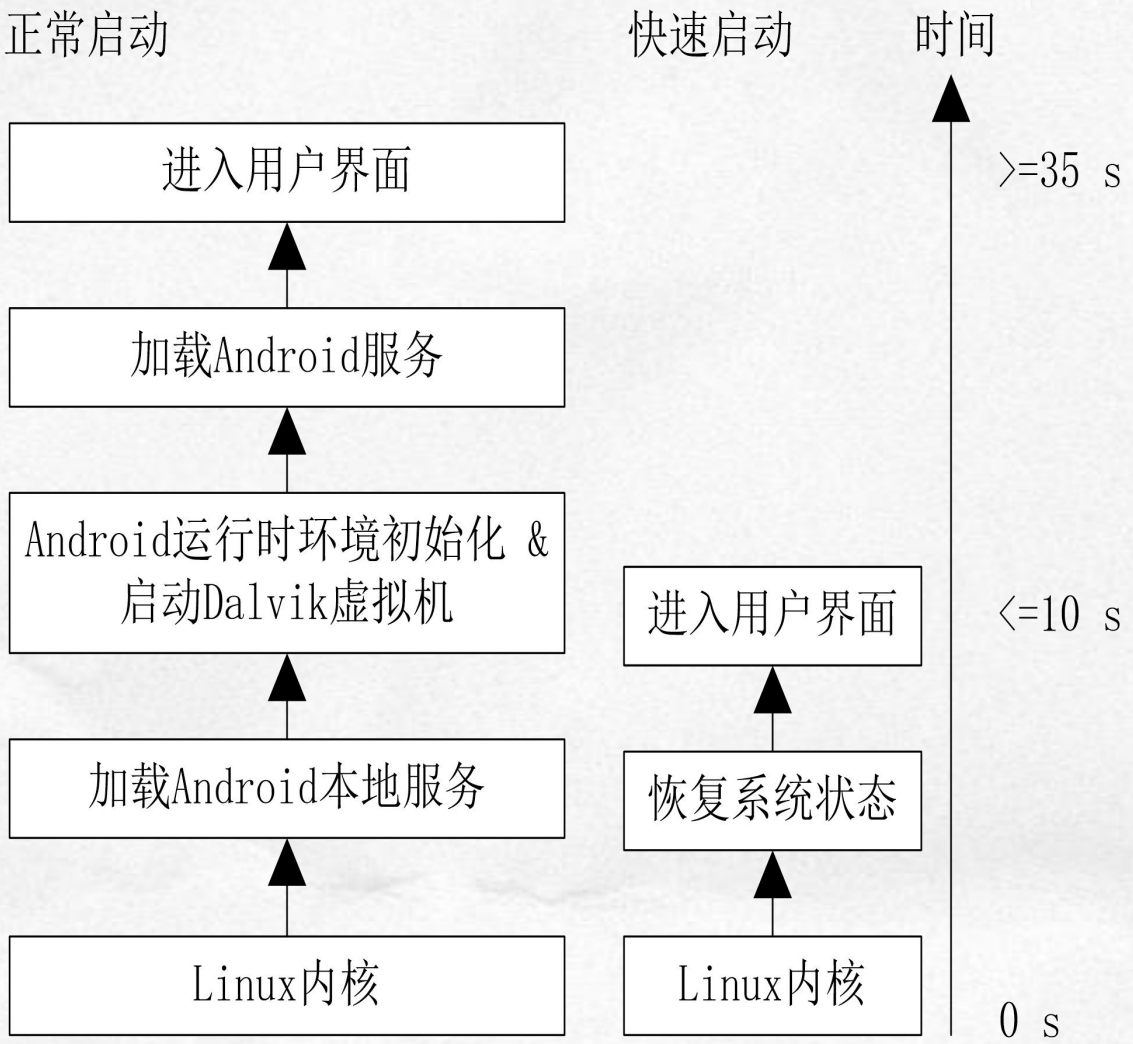
快速启动的方法

- **通用原理**　针对影响因素，逐项优化
- **具体措施**　没有完全通用的流程，只能针对具体应用优化
- **硬件优化**　提升处理器性能、存储容量、… → **通用，但成本？**
- **软件优化**　优化启动运行过程！→ 代码优化；**专用、延迟启动、待机、…**

对嵌入式系统

示例：Android的快速启动

制定全新的android启动过程



思考题

1、嵌入式软件运行过程有几个阶段？每个阶段的主要工作是什么？

2、什么是BSP？BSP与BIOS有什么异同？

3、嵌入式操作系统有那两种引导模式？比较两种引导模式的优缺点。

4、举例说明AI辅助的驱动程序编写过程

谢谢！